

When MovieBot provides an inconsistent response or does not understand the intent of the user, how do we debug the code that is running remotely? For situations such as this, we have to log numerous details: NLP scores, the dialogue responses, query results of the movie ticket database, etc. Then we have to manually analyse and do detective work to find out what could have gone wrong. And, that is painful.

▪ **State management**

Although serverless is inherently stateless, real-world applications invariably have to deal with state. Orchestrating a set of serverless functions becomes a significant challenge when there is a common context that has to be passed between them.

Any chatbot conversation represents a dialogue. It is important for the program to understand the entire conversation. For example, for the query, "Is 'Dunkirk' playing in Urvashi Theatre in Bengaluru tonight?" if the answer from MovieBot is "Yes", then the next query from the user could be, "Are two tickets available?" If MovieBot confirms this, the user could say, "Okay, book it." For this transaction to work, MovieBot should remember the entire dialogue, which includes the name of the movie, the theatre's location, the city, and the number of tickets to book. This entire dialogue represents a sequence of stateless function calls. However, we need to persist this state for the final transaction to be successful. This maintenance of state external to functions is a tedious task.

▪ **Vendor lock-in**

Although we talk about isolated functions that are executed independently, we are in practice tied to the SDK (software development kit) and the services provided by the serverless technology platform. This could result in vendor lock-in because it is difficult to migrate to other equivalent platforms.

Let's assume that we implement the MovieBot on the AWS Lambda platform using Python. Though the core logic of the bot is written as Lambda functions, we need to use other related services from the AWS platform for the chatbot to work, such as AWS Lex (for NLP), AWS API gateway, DynamoDB (for data persistence), etc. Further, the bot code may need to make use of the AWS SDK to consume the services (such as S3 or DynamoDB), and that is written using boto3. In other words, for the bot to be a reality, it needs to consume many more services from the AWS platform than just the Lambda function code written in plain Python. This results in vendor lock-in because it is harder to migrate the bot to other platforms.

▪ **Other challenges**

Each serverless function code will typically have third party library dependencies. When deploying the serverless function, we need to deploy the third party dependency packages as well, and that increases the deployment package size. Because containers are used underneath to execute the serverless functions, the increased deployment size increases the latency to start up and execute the serverless functions.

Further, maintaining all the dependent packages, versioning them, etc, is a practical challenge as well.

Another challenge is the lack of support for widely used languages from serverless platforms. For instance, as of May 2017, you can write functions in C#, Node.js (4.3 and 6.10), Python (2.7 and 3.6) and Java 8 on AWS Lambda. How about other languages like Go, PHP, Ruby, Groovy, Rust or any others of your choice? Though there are solutions to write serverless functions in these languages and execute them, it is harder to do so. Since serverless technologies are maturing with support for a wider number of languages, this challenge will gradually disappear with time.

Serverless is all about creating solutions without thinking or worrying about servers; think of it as just putting your code in the cloud and running it! Serverless is a game-changer because it shifts the way you look at how applications are composed, written, deployed and scaled. If you want significant agility in creating highly scalable applications while remaining cost-effective, serverless is what you need. Businesses across the world are already providing highly compelling solutions using serverless computing technologies. The applications serverless has range from chatbots to real-time stream processing from IoT (Internet of Things) devices. So it is not a question of if, but rather, when you will adopt the serverless approach for your business. 

References

- [1] 'Build Your Own Technology Radar', Neal Ford, http://nealford.com/memeagora/2013/05/28/build_your_own_technology_radar.html
- [2] 'Serverless Architectures', Martin Fowler, <https://martinfowler.com/articles/serverless.html>
- [3] 'Why the Fuss About Serverless?' Simon Wardley, <http://blog.gardeviance.org/2016/11/why-fuss-about-serverless.html>
- [4] 'Serverless Architectural Patterns and Best Practices', Amazon Web Services, <https://www.youtube.com/watch?v=b7UMoc1iUYw>

Serverless technologies

- AWS Lambda: <https://aws.amazon.com/lambda/>
- Azure Functions: <https://functions.azure.com/>
- Google Cloud Functions: <https://cloud.google.com/functions/>
- Apache OpenWhisk: <https://github.com/openwhisk>
- Serverless framework: <https://github.com/serverless/serverless>
- Fission: <https://github.com/fission/fission>
- Hyper.sh: <https://github.com/hyperhq/>
- Funktion: <https://funktion.fabric8.io/>
- Kubeless: <http://kubeless.io/>

By: Ganesh Samarthayam, Manoj Ganapathi and Srushit Repakula

The authors work at CodeOps Technologies, which is a software technology, consulting and training company based in Bengaluru. CodeOps is the organiser of the upcoming India Serverless Summit, scheduled on October 27, 2017. Please check www.codeops.tech for more details.